Basic Matrix Multiplication in Python, Java and C: How Time and Memory Grow with n

Pablo Castillo Garcerá

Big Data, Individual Assignment 1, 2026-2027

Code, data and results: https://github.com/Pablo5250/matrix-multiplication (folder assignment1/)

1. The problem

Matrix multiplication is one of the most used operations in data science: linear regression, neural networks and many other methods are, inside, millions of matrix products. To compute C = A × B for two n × n matrices, each element is c[i][j] = Σk a[i][k] × b[k][j]. The basic algorithm uses three nested loops (i, j, k), so it does n^3 multiply-adds: its time complexity is $O(n^3)$ [1], while storing the three matrices needs only $3 \times 8n^2$ bytes. Doubling n means about 8 times more time but only 4 times more memory, so the cost grows very fast.

Research question. How does the matrix size n affect the runtime and the memory use of the same basic triple-loop algorithm written in Python, Java and C, on one machine?

Hypotheses (before measuring): H1, time grows like n^3 in the three languages; H2, Python is much slower because it is interpreted; H3, C is the fastest, with Java close; H4, C uses about the memory of the matrices, while Java and Python use more.

How the comparison was made fair

- **Same algorithm:** the three programs use the same i-j-k loops and 64-bit doubles, with no optimised library. Only the storage differs: a list of lists of float objects in Python, one array per row in Java (`double[][]`), and one contiguous block in C.
- **Same inputs:** A and B (uniform values in [0, 1), NumPy PCG64, seed 42 + n) are created once and saved to binary files that the three programs read, so they multiply exactly the same numbers.
- **Correct before fast:** unit tests (2×2 and 3×3 by hand, A × I = A, A × 0 = 0) and a comparison with NumPy for n = 16, 64, 128 (relative tolerance 10^{-9}, absolute 10^{-12}: the expected rounding error is about n × 1.1×10^{-16}, below 10^{-13} here, so this margin still catches any real bug). All passed, and the three languages gave bit-identical results.
- **Timer:** only the call multiply(A, B), including creating C; reading files is not timed. Each configuration runs in a new process: warm-up runs (3 in Java for the JIT compiler, 1 in C and Python), then 5 measured runs. We report the median and the IQR (spread of the middle half).
- **Budget set before running:** 1200 s per configuration and 4 GB. A size predicted to exceed it is skipped, not invented. **Memory:** peak working set of the whole process, read from Windows every 10 ms with psutil.

Table 1. Test machine and software.

Item	Value
CPU	AMD Ryzen 9 5900X 12-Core Processor
Cores	12 physical, 24 logical
RAM	34.3 GB
Operating system	Windows 11 (10.0.26300)
Python	CPython 3.14.3 (NumPy 2.4.3, psutil 7.2.2, matplotlib 3.10.9)
Java	OpenJDK 64-Bit Server VM (build 25.0.1+8-27, mixed mode, sharing); options -Xmx4g
C compiler	gcc (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r1) 16.2.0; flags -O2 -std=c11
Threads	kernels single-threaded; no BLAS; the JVM also runs its own JIT and GC threads

2. Results

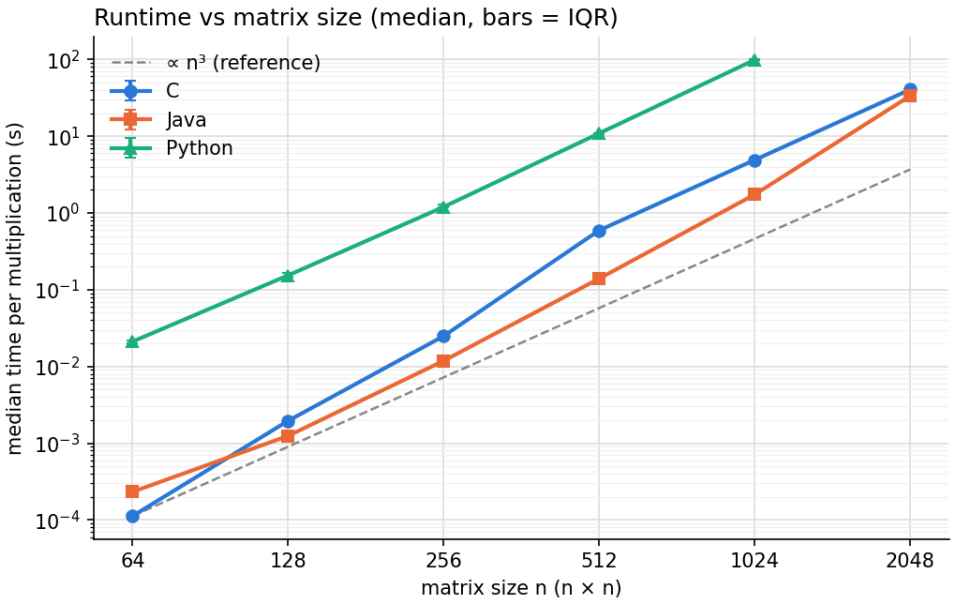


Figure 1. Median time of one multiplication vs n (log-log axes). The dashed line grows like n³.

All lines grow roughly like the n³ line, so **H1 holds**, although C and Java become a few times slower per operation at large n (Table 2), because of the cache effect explained below. Python is far above the others: 18 to 190 times slower than C (99 s at n = 1024). Python at n = 2048 was skipped, because the predicted time (about 4,700 s) was over the budget; there were no timeouts or memory failures. **H2 holds.** The reason is the interpreter: each c[i][j] += a[i][k] × b[k][j] runs several bytecode instructions and creates a new float object, while C and Java run a few machine instructions. The surprise is that **Java was faster than C** at every n ≥ 128, so **H3 fails** at these sizes.

Table 2. Median and IQR of 5 measured runs per configuration. "ns / iteration" = median / n³; "× C" = median / median of C at the same n.

Language	n	Runs	Median time	IQR	ns / iteration	× C	Peak memory (MiB)
C	64	5	0.112 ms	0.002 ms	0.43	1.00	1.9
C	128	5	1.953 ms	0.023 ms	0.93	1.00	4.9
C	256	5	24.8 ms	0.494 ms	1.48	1.00	6.0
C	512	5	591.6 ms	1.316 ms	4.41	1.00	10.4
C	1024	5	4.90 s	13.8 ms	4.56	1.00	28.4
C	2048	5	40.48 s	222.5 ms	4.71	1.00	100.4
Java	64	5	0.232 ms	0.029 ms	0.89	2.07	45.6
Java	128	5	1.246 ms	0.018 ms	0.59	0.64	46.9
Java	256	5	11.8 ms	0.098 ms	0.70	0.48	51.1
Java	512	5	139.3 ms	0.239 ms	1.04	0.24	69.6
Java	1024	5	1.73 s	142.9 ms	1.62	0.35	138.1
Java	2048	5	33.45 s	441.9 ms	3.89	0.83	361.5
Python	64	5	21.1 ms	2.032 ms	80.46	187.66	14.7
Python	128	5	153.7 ms	15.7 ms	73.27	78.68	16.3
Python	256	5	1.20 s	98.6 ms	71.37	48.20	22.9
Python	512	5	10.87 s	449.0 ms	80.99	18.38	44.9
Python	1024	5	98.61 s	1.85 s	91.84	20.14	135.5

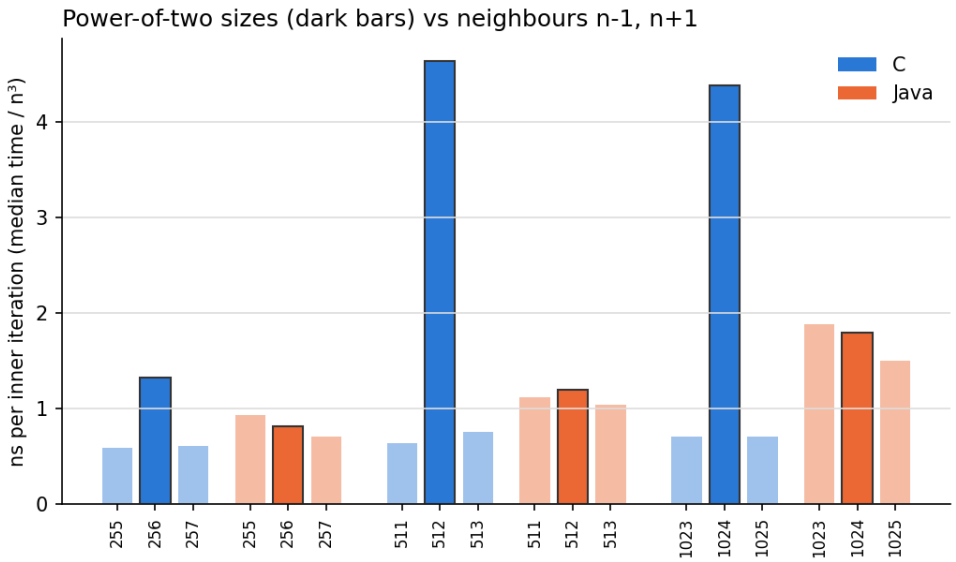


Figure 2. Extra experiment: time per inner iteration at n − 1, n and n + 1. Dark bars are the powers of two.

All the tested sizes are powers of two, so we ran C and Java again at n − 1 and n + 1. **In C, n = 512 and 1024 are 6-7 times slower than their neighbours** (4.6 ns vs 0.6-0.8 ns per iteration); Java hardly changes, and at the neighbour sizes C is the fastest. The most likely cause is **data locality** [2], [3]: the inner loop reads a column of B. In C the rows are one block, so b[k][j] and b[k+1][j] are exactly 8n bytes apart (4096 bytes at n = 512). On this CPU those addresses all go to the same cache set, which holds only 8 lines [4], so the column has to be read again from slower memory every time (conflict misses). In Java each row is a separate object with a header, the distance is not a power of two, and the problem mostly disappears. We did not count cache misses directly, but the n ± 1 results fit this explanation. The code is correct; the slowdown comes from the basic algorithm and the memory layout.

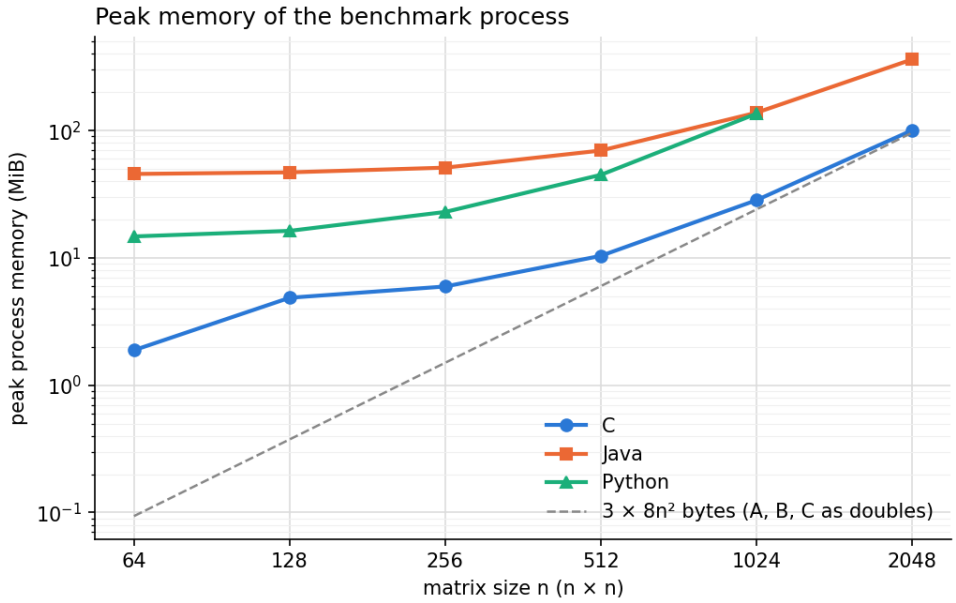


Figure 3. Peak memory of the process vs n. The dashed line is the size of A, B and C as doubles (3 × 8n² bytes).

H4 holds. C stays close to the size of the data (100 MiB for 96 MiB at n = 2048). Java starts at about 45 MiB for the JVM itself and reaches 361 MiB, probably because each run creates a new result and the garbage collector frees old ones later. Python uses 135 MiB for 24 MiB of data at n = 1024: each number is an object (24 bytes) plus a pointer (8 bytes), about 4 times more than C.

3. Conclusions and future work

On this machine and with these settings:

- The basic algorithm grows like n^3 in the three languages, but the cost of one operation is not fixed: it depends on how the data uses the cache.
- Python is 18-190 times slower than C; its largest size within the budget is n = 1024.
- Java beat C only at power-of-two sizes, because of cache conflicts in C's memory layout. At n ± 1, C is the fastest. One language is not “always faster”: the data layout matters as much.
- C uses about the memory of the data; Java adds the JVM and garbage; Python needs about 4 times more memory per number.

Limitations: one computer, 5 runs per configuration (IQR below 3% of the median in most cases, at most 12%), memory sampled every 10 ms (very short runs may be underestimated), and no hardware counters to measure cache misses directly.

Future work (next assignments): the cache problem comes from reading B by columns. Changing only the loop order to i-k-j makes all accesses sequential, which the course notes show can be up to 6 times faster at n = 1024 [5]. Blocking (tiling) should reuse the cache even better, and sparse formats such as CRS should save time and memory when most values are zero [5]. After that, the work can be split across cores (Assignment 3) and machines (Assignment 4).

References and materials

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., 2007.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.

[4] AMD, “Software Optimization Guide for AMD Family 19h Processors,” pub. 56665, 2020.

[5] Big Data course notes, “Experiments: Monitoring and Performance Engineering,” Grado en Ciencia e Ingeniería de Datos, 2026.

Materials. https://github.com/Pablo5250/matrix-multiplication, folder assignment1/: source code, config.json, README.md with the exact commands (run_all.bat repeats the whole study), raw measurements (results/raw/times.csv), summary tables, analysis scripts and figures. The input matrices are recreated by data/generate_inputs.py and checked with SHA-256 (data/manifest.json).

Use of generative AI. I used Claude (Anthropic) as a support tool to help write the code and draft the text of this report. I reviewed all the code and the text, ran every test and measurement on my own computer, and checked the results. I am responsible for the code, the data and the conclusions.